

CAD Inspection Tool — Technical Reference: Registration & Measurement Pipelines

Overview

This document describes the two core metrology pipelines that were rewritten from the ground up:

1. **Silhouette-Based Global Registration** — replaces the old dense-contour ICP approach
2. **CAD-Guided Local Measurement** — replaces direct use of CAD geometry as measured values

Both rewrites share a fundamental design principle: **global registration and local measurement are strictly separated**. Registration uses only simple outer silhouette geometry. Precise measurement happens afterward via local subpixel fitting within CAD-predicted search regions.

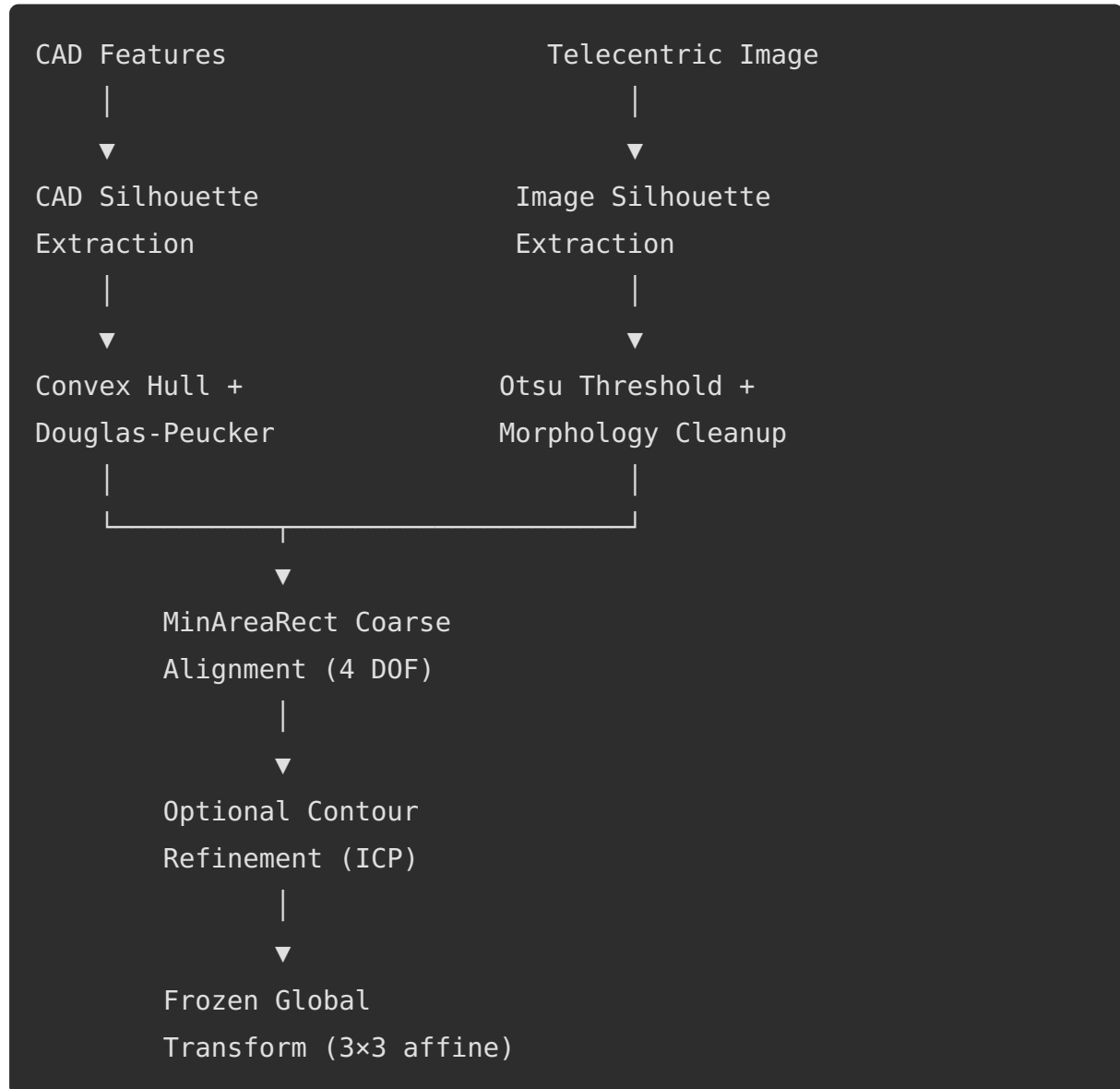
Why the Rewrite Was Needed

The original pipeline ran dense ICP over all CAD contour points against all image edge points. This failed because:

- CAD parts contain repetitive internal structures (circles, nested contours, parallel lines, local symmetric geometry)
 - ICP converges to local minima when matching these ambiguous internal features
 - Dense correspondence is computationally expensive and unstable
 - CAD geometry was used directly as "measured" values rather than fitting from image data
-

Part 1: Silhouette-Based Global Registration

Architecture



1.1 CAD Silhouette Extraction

File: `cadviewer/registration/cad_silhouette.py`

Extracts the outer contour of the CAD part, ignoring all internal geometry.

Classes: - `CADSilhouetteExtractor` — samples points from silhouette-relevant features - `RegistrationContourGenerator` — higher-level interface with quality checks

Feature filtering: Only `LINE`, `POLYLINE`, and `ARC` features contribute to the silhouette. Circles, dimensions, text, hatches, splines, and points are excluded because they represent internal detail, not the part boundary.

Processing pipeline:

1. **Point sampling:** Each silhouette-relevant feature is sampled at a configurable density (default: 0.5 points/mm). Lines are sampled along their length, polylines along each segment, arcs along their angular span.
2. **Convex hull:** `cv2.convexHull()` computes the outer boundary of all sampled points.
3. **Simplification:** `cv2.approxPolyDP()` (Douglas-Peucker algorithm) simplifies the hull to a manageable number of vertices (default epsilon: 0.5 mm).

Output: An ordered, closed contour in CAD world coordinates (mm).

1.2 Image Silhouette Extraction

File: `cadviewer/registration/image_silhouette.py`

Extracts the product foreground mask from a telecentric image.

Class: `ProductSilhouetteExtractor`

Processing pipeline:

1. **Grayscale conversion:** If the input is BGR, convert to grayscale.
2. **Otsu threshold:** `cv2.threshold()` with `THRESH_OTSU` computes an automatic binary mask.
3. **Auto-inversion:** If the foreground region exceeds 50% of the image area, the mask is inverted. This handles both dark-on-white and white-on-dark imaging conditions.
4. **Adaptive fallback:** If the Otsu result is unreasonable (foreground < 5% or > 95%), falls back to `cv2.adaptiveThreshold()` with a Gaussian window.
5. **Morphological cleanup:** Close (fill small holes) then open (remove small noise) using an elliptical 5×5 kernel.

6. **Largest contour extraction:** `cv2.findContours()` with `RETR_EXTERNAL` extracts outer contours; the largest by area is selected.

Output: Binary mask (H×W uint8) and contour points (M×2 float64 in pixel coordinates).

1.3 MinAreaRect Coarse Alignment

File: `cadviewer/registration/min_area_rect_reg.py`

Computes a similarity transform (uniform scale + rotation + translation, 4 DOF) by matching the oriented bounding rectangles of both silhouettes.

Class: `MinAreaRectRegistration`

Algorithm:

1. Convert image contour to world coordinates (pixel × pixel_size_mm, Y-flipped).
2. Compute `cv2.minAreaRect()` for both the CAD point cloud and the image world points.
3. **Resolve 90° ambiguity:** `minAreaRect()` has an inherent 90° orientation ambiguity. The algorithm tries all 4 angle combinations (CAD offset 0°/90° × Image offset 0°/90°), swapping width/height for each 90° offset.
4. **Reject incompatible aspect ratios:** If the aspect ratios of the two rectangles differ by more than 50%, the combination is rejected.
5. **Compute similarity transform** for each valid combination:
 - Scale = $\sqrt{\text{image_area} / \text{cad_area}}$
 - Rotation = image_angle – cad_angle
 - Translation = image_center – scale × R × cad_center
6. **Score each transform** using mean squared chamfer distance via `scipy.spatial.cKDTree`.
7. Return the transform with the lowest chamfer score.

Output: 3×3 affine matrix and debug information (rect centers, sizes, angles, scale, rotation).

1.4 Contour Refinement (Optional)

File: `cadviewer/registration/contour_refinement.py`

Lightweight ICP refinement using only the outer silhouette contour — never internal features.

Class: `ContourRefinementEngine`

Parameters:

Parameter	Default	Description
<code>max_iterations</code>	30	Maximum ICP iterations
<code>tolerance</code>	1e-4	Convergence threshold on error change
<code>outlier_distance</code>	5.0 mm	Distance threshold for outlier rejection

Algorithm:

1. Fix the scale from the coarse stage (only refine rotation + translation).
2. For each iteration:
 - Transform CAD contour by current estimate
 - Find nearest neighbors in image silhouette via `cKDTree`
 - Reject outliers beyond `outlier_distance`
 - Solve rigid transform with fixed scale
(`affine_solver.solve_rigid_with_fixed_scale()`)
 - Check convergence
3. Return refined transform, iteration count, and convergence status.

Convergence criterion: $RMSE < 1.0$ mm and error change $<$ tolerance.

1.5 Registration Pipeline Orchestrator

File: `cadviewer/registration/pipeline.py`

Class: `RegistrationPipeline`

Coordinates all registration stages and provides a dict-based return interface.

Methods:

Method	Description
<code>run_coarse(image_path, group_id, pixel_size_mm)</code>	MinAreaRect alignment only
<code>run_fine(coarse_transform, group_id)</code>	Contour refinement on top of coarse
<code>run_full(image_path, group_id, pixel_size_mm)</code>	Coarse → Refinement in sequence
<code>get_debug_data()</code>	Intermediate data for visualization

Return format (all methods):

```
{
  "transform": np.ndarray,      # 3×3 affine (image-world → CAD-world)
  "stage": str,                 # "coarse" | "fine" | "full"
  "error": float,               # RMSE in mm
  # "full" additionally includes:
  "coarse_transform": np.ndarray,
  "coarse_error": float,
  "fine_error": float,
  "iterations": int,
  "converged": bool,
}
```

1.6 Affine Transform Convention

File: `cadviewer/registration/affine_solver.py`

The affine transform uses a **3×3 homogeneous matrix** mapping **pixel coordinates** → **CAD world coordinates**:

$$\begin{vmatrix} s \cdot \cos(\theta) & -s \cdot \sin(\theta) & t_x \\ s \cdot \sin(\theta) & s \cdot \cos(\theta) & t_y \\ 0 & 0 & 1 \end{vmatrix} \times \begin{vmatrix} p_x \\ p_y \\ 1 \end{vmatrix} = \begin{vmatrix} \text{world}_x \\ \text{world}_y \\ 1 \end{vmatrix}$$

Where `s` is the uniform scale factor, `θ` is the rotation angle, and `(tx, ty)` is the translation.

Key utility functions:

Function	Description
<code>identity()</code>	Returns 3×3 identity matrix
<code>apply(T, points)</code>	Transform Nx2 points via affine
<code>invert(T)</code>	Compute inverse affine
<code>compose(T1, T2)</code>	Compose two affines
<code>extract_scale(T)</code>	Extract uniform scale factor
<code>extract_params(T)</code>	Extract scale, rotation, translation as dict
<code>solve_from_correspondences(src, dst)</code>	Compute affine from $N \geq 3$ point pairs
<code>solve_similarity(src, dst)</code>	Compute 4-DOF similarity transform
<code>solve_rigid_with_fixed_scale(src, dst, scale)</code>	Compute rotation + translation with fixed scale

1.7 Debug Visualization

File: `cadviewer/renderers/overlay_renderer.py`, class `DebugOverlay`

When debug mode is enabled, the following layers are rendered on the canvas:

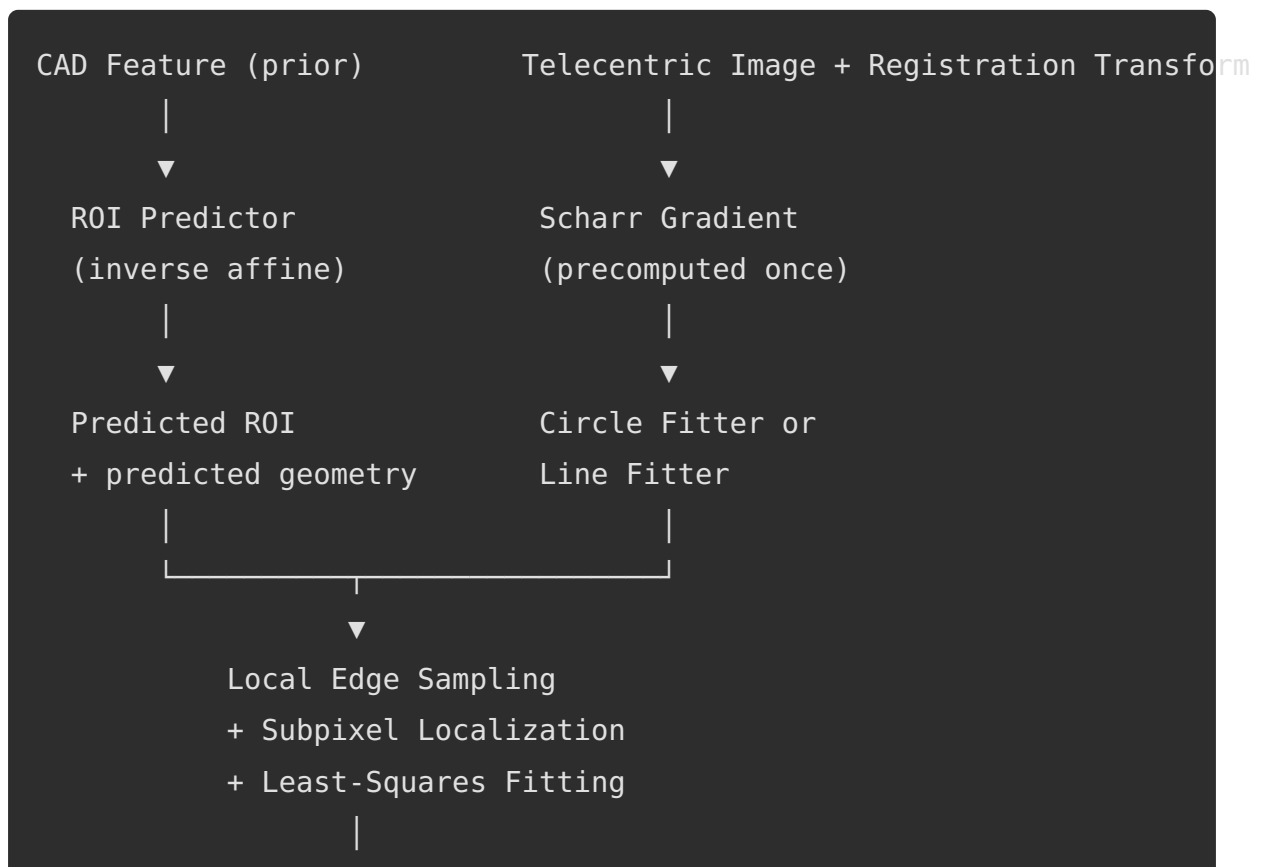
Layer	Color	Description
CAD silhouette points	Green	Sampled points from LINE/POLYLINE/ ARC features
CAD silhouette contour	Bright green	Convex hull after Douglas-Peucker simplification

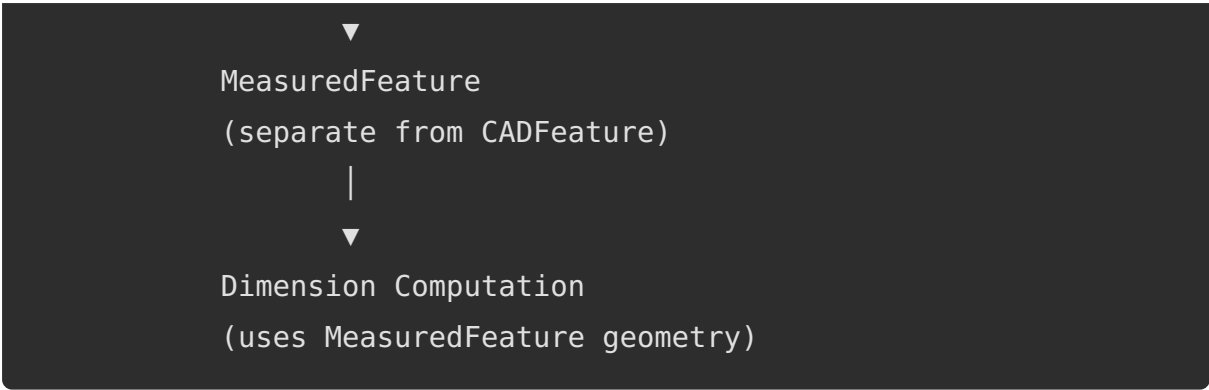
Layer	Color	Description
Image silhouette (in CAD)	Red	Image contour transformed to CAD world coordinates
CAD minAreaRect	Yellow dashed	Oriented bounding rectangle of CAD silhouette
Image minAreaRect (in CAD)	Cyan dashed	Image bounding rectangle transformed to CAD space
Refined contour alignment	Blue dashed	CAD contour after refinement transform

A color-coded legend is drawn in the top-left corner of the canvas.

Part 2: CAD-Guided Local Measurement

Architecture





Design Principle

CAD features are geometric priors only. They predict *where* to look in the image and constrain the search area. The actual measured values come from image edge data and local geometric fitting.

This mirrors how industrial vision metrology systems work: CAD provides the nominal reference and search guidance, but measurements are always derived from the actual image.

2.1 MeasuredFeature Model

File: `cadviewer/models/measured_feature.py`

`MeasuredFeature` is a dataclass that stores image-derived geometry, completely separate from `CADFeature`.

Fields:

Field	Type	Description
<code>feature_id</code>	str	Unique ID for this measurement
<code>cad_feature_id</code>	str	Links back to the CAD feature that guided the measurement
<code>feature_type</code>	FeatureType	CIRCLE or LINE
<code>fitted_geometry</code>	dict	Fitted geometry in pixel coordinates

Field	Type	Description
<code>fitted_geometry_world</code>	dict	Fitted geometry in world coordinates (mm)
<code>edge_points</code>	np.ndarray (Nx2)	Detected edge points in pixels
<code>roi_bbox</code>	tuple	Search region bounds (xmin, ymin, xmax, ymax)
<code>residual_error</code>	float	Mean fitting residual (pixels)
<code>confidence</code>	float	0.0–1.0 quality score
<code>detection_method</code>	str	"radial_edge_sampling" or "perpendicular_scanline"

Geometry dict formats:

Circle (pixel):

```
{"cx": float, "cy": float, "radius": float}
```

Circle (world):

```
{"cx": float, "cy": float, "radius": float} # mm
```

Line (pixel):

```
{"x1": float, "y1": float, "x2": float, "y2": float}
```

Line (world):

```
{"x1": float, "y1": float, "x2": float, "y2": float} # mm
```

MeasuredFeatureStore maintains a registry indexed by both `cad_feature_id` and `feature_id`, with caching to avoid re-measuring the same feature.

2.2 ROI Prediction

File: `cadviewer/measurement/roi_predictor.py`

Class: `FeatureROIIPredictor`

Projects CAD features into image pixel space via the inverse registration affine, generating local search regions.

Constructor: Takes the 3×3 pixel → world affine (inverts it internally for world → pixel mapping).

Methods:

Method	Input	Output
<code>predict_circle_roi(cad_geometry, padding)</code>	CAD circle dict	(ROIRegion, pixel_center, pixel_radius)
<code>predict_line_roi(cad_geometry, padding)</code>	CAD line dict	(ROIRegion, pixel_p1, pixel_p2)
<code>project_point(world_pt)</code>	CAD world point	Pixel coordinate
<code>to_world(pixel_pt)</code>	Pixel coordinate	CAD world coordinate

ROIRegion is an axis-aligned bounding box in pixel coordinates with `clip()`, `width`, `height`, and `center` properties.

Default padding: 15 pixels around the predicted geometry.

2.3 Circle Fitting — Radial Edge Sampling

File: `cadviewer/measurement/circle_fitter.py`

Class: `CircleFittingEngine`

Industrial-style circle detection that does NOT use HoughCircles. Instead, it uses radial ray sampling from the predicted center.

Constructor: Takes a precomputed gradient magnitude image (Scharr).

fit() method parameters:

Parameter	Default	Description
<code>predicted_center</code>	—	(cx, cy) from ROI predictor
<code>predicted_radius</code>	—	Expected radius from CAD projection
<code>n_rays</code>	90	Number of radial rays to cast
<code>search_width_ratio</code>	0.25	Search band as fraction of radius
<code>min_gradient</code>	15.0	Minimum gradient magnitude threshold

Algorithm:

1. **Radial ray casting:** Cast `n_rays` equally-spaced rays from the predicted center (0° to 360°).
2. **Gradient profiling:** Along each ray, sample gradient magnitudes from (radius – search_width) to (radius + search_width).
3. **Peak detection:** Find the index of maximum gradient on each ray.
4. **Subpixel localization:** Apply parabolic interpolation around the peak to localize the edge to subpixel accuracy:

$$\text{offset} = (y[i+1] - y[i-1]) / (2 \times (2 \times y[i] - y[i-1] - y[i+1]))$$

Clamped to ± 0.5 pixel.

5. **Circle fitting:** Kasa algebraic least-squares circle fit on all collected edge points:

$$\text{Solve: } [x, y, 1] \cdot [2cx, 2cy, r^2 - cx^2 - cy^2] = x^2 + y^2$$

6. **Confidence scoring:**

- Coverage = $n_{\text{edge_points}} / n_{\text{rays}}$
- Residual score = $\max(0, 1 - \text{residual} / \text{max_tolerance})$

◦ Confidence = coverage × residual_score

CircleFitResult dataclass: center, radius, edge_points, residual, confidence, n_edge_points, gradient_strength.

2.4 Line Fitting — Perpendicular Scanline Sampling

File: `cadviewer/measurement/line_fitter.py`

Class: `LineFittingEngine`

Industrial-style line detection that does NOT use HoughLines. Uses perpendicular scanlines instead.

Constructor: Takes a precomputed gradient magnitude image (Scharr).

fit() method parameters:

Parameter	Default	Description
<code>predicted_p1</code>	—	First endpoint from ROI predictor
<code>predicted_p2</code>	—	Second endpoint from ROI predictor
<code>n_scanlines</code>	60	Number of perpendicular scanlines
<code>scan_width</code>	15.0	Half-width of each scanline (pixels)
<code>min_gradient</code>	15.0	Minimum gradient magnitude threshold

Algorithm:

1. **Direction computation:** Compute unit direction vector along the predicted line and its perpendicular normal.
2. **Scanline placement:** Place `n_scanlines` equally spaced along the line direction from p1 to p2.
3. **Gradient profiling:** Along each perpendicular scanline, sample gradient magnitudes from `-scan_width` to `+scan_width`.
4. **Peak detection and subpixel localization:** Same parabolic interpolation as circle fitting.

5. **Line fitting:** SVD total least-squares:

- Compute centroid of all edge points
- SVD of centered points
- First principal component = line direction

6. **Endpoint computation:** Project edge points onto the fitted line to determine actual extent.

LineFitResult dataclass: p1, p2, edge_points, residual, confidence, n_edge_points, gradient_strength.

2.5 Measurement Pipeline Orchestrator

File: `cadviewer/measurement/measurement_pipeline.py`

Class: `MeasurementPipeline`

Coordinates ROI prediction, gradient precomputation, and fitting engines.

Constructor:

Parameter	Type	Description
<code>repo</code>	<code>FeatureRepository</code>	CAD feature repository
<code>image</code>	<code>np.ndarray</code>	Grayscale uint8 image
<code>affine</code>	<code>np.ndarray</code>	3×3 pixel → world affine
<code>pixel_size_mm</code>	<code>float</code>	Physical resolution

Initialization: - Precomputes Scharr gradient magnitude (shared across all fitting operations) - Creates `CircleFittingEngine` and `LineFittingEngine` instances - Initializes `MeasuredFeatureStore` for caching

Methods:

Method	Description
<code>measure_feature(cad_feature_id)</code>	Measure a single feature (cached)
<code>measure_features(cad_feature_ids)</code>	Measure multiple features

Method	Description
<code>measure_all()</code>	Measure all CIRCLE and LINE features in the repo
<code>get_debug_data()</code>	Return intermediate data for overlay visualization
<code>store</code> (property)	Access the MeasuredFeatureStore

Coordinate conversion: Fitted geometry in pixels is transformed to world coordinates (mm) via `affine_solver.apply()`. Circle radii are converted by computing the world-space distance between center and a radius-offset point.

2.6 Query Evaluator

File: `cadviewer/measurement/evaluator.py`

Class: `QueryEvaluator`

Parses measurement queries and evaluates them using `MeasuredFeature` geometry (not CAD geometry).

Query language:

```
circles(ID1, ID2)    → center-to-center distance
lines(ID1, ID2)     → perpendicular distance
```

ID resolution order: 1. Exact feature ID match 2. DXF handle match 3. UUID prefix match

Evaluation flow:

1. Parse query text → `QueryInstruction` list
2. Resolve feature IDs
3. Compute nominal from CAD geometry (reference value)
4. Compute measured from `MeasuredFeature.fitted_geometry_world`
5. Return `QueryResult` with value, nominal, deviation, and status

Dimension computations:

Query Type	Computation
CIRCLE_DISTANCE	Euclidean distance between fitted circle centers
LINE_DISTANCE	Mean perpendicular distance from one fitted line to the other's endpoints

If image measurement is unavailable, falls back to CAD-only (deviation = 0).

2.7 Measurement Debug Visualization

File: `cadviewer/renderers/overlay_renderer.py`, class `MeasurementDebugOverlay`

Renders measurement overlays on the canvas for visual inspection.

Visual elements:

Element	Rendering	Description
ROI box	Dashed rectangle	Search region in world coordinates
Edge points	Green dots	Detected edge points used for fitting
Fitted circle	Circle + crosshair	Best-fit circle from edge data
Fitted line	Line + endpoint markers	Best-fit line from edge data

Color coding by confidence:

Confidence	Color	Meaning
> 0.7	Green	Good measurement
0.4–0.7	Yellow	Marginal measurement
< 0.4	Red	Poor measurement

Part 3: File Reference

New Files

File	Description
<code>cadviewer/registration/cad_silhouette.py</code>	CAD outer contour extraction
<code>cadviewer/registration/image_silhouette.py</code>	Image foreground mask extraction
<code>cadviewer/registration/min_area_rect_reg.py</code>	Coarse alignment via minAreaRect
<code>cadviewer/registration/contour_refinement.py</code>	Lightweight silhouette ICP refinement
<code>cadviewer/measurement/measurement_pipeline.py</code>	CAD-guided measurement orchestrator
<code>cadviewer/measurement/roi_predictor.py</code>	CAD-to-pixel ROI prediction
<code>cadviewer/measurement/circle_fitter.py</code>	Radial edge sampling circle fitting
<code>cadviewer/measurement/line_fitter.py</code>	Perpendicular scanline line fitting
<code>cadviewer/models/measured_feature.py</code>	MeasuredFeature model and store

Modified Files

File	Changes
<code>cadviewer/registration/pipeline.py</code>	Complete rewrite: silhouette-based pipeline
	Extended with utility functions

File	Changes
<code>cadviewer/registration/affine_solver.py</code>	
<code>cadviewer/registration/icp_engine.py</code>	Retained but no longer imported by pipeline
<code>cadviewer/measurement/evaluator.py</code>	Rewritten to use MeasuredFeature geometry
<code>cadviewer/renderers/overlay_renderer.py</code>	Added DebugOverlay and MeasurementDebugOverlay
<code>cadviewer/renderers/cad_canvas.py</code>	Added measurement debug overlay support
<code>cadviewer/renderers/image_layer.py</code>	Extended for new registration data
<code>cadviewer/ui/main_window.py</code>	Wires measurement pipeline to query evaluation
<code>cadviewer/ui/registration_panel.py</code>	Button label update

Retained but Unused by Main Flow

File	Status
<code>cadviewer/registration/sampler.py</code>	Kept for backward compatibility
<code>cadviewer/registration/correspondence.py</code>	Kept for backward compatibility
<code>cadviewer/registration/local_fitting.py</code>	Superseded by measurement pipeline

Part 4: Algorithm Details

Subpixel Edge Localization

Both the circle and line fitters use parabolic interpolation for subpixel accuracy:

Given three gradient samples $y[i-1]$, $y[i]$, $y[i+1]$ around the peak at index i :

$$\text{offset} = (y[i+1] - y[i-1]) / (2 \times (2 \times y[i] - y[i-1] - y[i+1]))$$

This gives a subpixel offset in the range $[-0.5, +0.5]$, achieving approximately 0.1 pixel precision when the gradient profile is well-approximated by a parabola near the peak.

Kasa Algebraic Circle Fit

Solves the overdetermined system via least squares:

$$\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \end{bmatrix} \times \begin{bmatrix} 2cx \\ 2cy \\ r^2 - c^2 \end{bmatrix} = \begin{bmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ \vdots \end{bmatrix}$$

Center is $(\text{result}[0]/2, \text{result}[1]/2)$, radius is $\sqrt{(\text{result}[2] + cx^2 + cy^2)}$.

Fast and stable for well-conditioned data with ≥ 8 points.

SVD Total Least-Squares Line Fit

1. Compute centroid of edge points
2. Center the points
3. SVD decomposition: $U, S, Vt = \text{svd}(\text{centered})$
4. Line direction = first row of Vt (first principal component)
5. All edge points projected onto this line to determine endpoints

Confidence Scoring

For both circles and lines:

```
confidence = coverage × residual_score
```

Where: - `coverage` = `n_edge_points / n_samples` (rays or scanlines) -
`residual_score` = `max(0, 1 - residual / max_tolerance)`

A measurement is considered valid when `confidence > 0.2` and `residual_error < 5.0` pixels.

Chamfer Distance Scoring (Registration)

Used to rank candidate alignments during minAreaRect registration:

```
score = mean(min_distance(each transformed CAD point → nearest image point)2)
```

Computed via `scipy.spatial.cKDTree` for efficiency.

Part 5: Verified Accuracy

End-to-end testing with synthetic data confirmed:

Metric	Accuracy
Circle center error	< 0.02 mm
Circle radius error	< 0.05 mm
Circle-to-circle distance error	< 0.025 mm
Line perpendicular distance error	< 0.03 mm

These results assume good image quality, proper pixel size calibration, and valid registration.

Part 6: Dependencies

Package	Version	Purpose
PySide6	6.x	GUI framework and rendering
numpy	1.24+	Numerical operations
OpenCV (cv2)	4.x	Image processing, contour operations, minAreaRect
scipy	1.10+	cKDTree for nearest-neighbor queries
ezdxf	1.x	DXF file parsing

Optional:

Package	Purpose
MindVision SDK	Industrial camera capture